47 TYPEORM QUERY EXAMPLES — PRODUCT + CATEGORY (NESTJS + POSTGRESQL)

Simple, copy-ready TypeORM queries in one place, using a Product entity (id, name, price) that belongs to a Category. For each example: the TypeORM code and the SQL TypeORM generates for it.

The same code works on ANY PostgreSQL host — Docker, local install, Supabase, Neon, RDS. Only the connection config (DATABASE_URL + ssl) changes, never the query methods (theory/42).

SETUP — ENTITIES, MODULE, SERVICE

```
// category.entity.ts
@Entity('categories')
export class Category {
  @PrimaryGeneratedColumn() id!: number;
  @Column() name!: string;

  @OneToMany(() => Product, (product) => product.category)
  products!: Product[];                 // one category -> many products
}

// product.entity.ts
@Entity('products')
export class Product {
  @PrimaryGeneratedColumn() id!: number;
  @Column() name!: string;
  @Column('int') price!: number;

  @ManyToOne(() => Category, (category) => category.products, { nullable: true })
  category?: Category;                  // many products -> one category
                                        // creates a "categoryId" foreign key column
}

// product.module.ts
@Module({
  imports: [TypeOrmModule.forFeature([Product, Category])],
  providers: [ProductService],
  controllers: [ProductController],
})
export class ProductModule {}

// product.service.ts
@Injectable()
export class ProductService {
  constructor(
    @InjectRepository(Product) private productRepo: Repository<Product>,
  ) {}
  // every example below goes inside methods of this class
}

Tables synchronize creates (theory/44):
  categories(id SERIAL PK, name varchar)
```

```
  products(id SERIAL PK, name varchar, price int,
           categoryId int NULL REFERENCES categories(id))
```

1. BASIC CRUD QUERIES

```
// CREATE
await this.productRepo.save({
  name: 'Laptop',
  price: 50000,
});
SQL: INSERT INTO "products"("name", "price", "categoryId")
     VALUES ($1, $2, DEFAULT) RETURNING "id"
     -- params: ['Laptop', 50000]

// READ ALL
await this.productRepo.find();
SQL: SELECT "id", "name", "price" FROM "products"

// READ BY ID
await this.productRepo.findOneBy({ id: 1 });
SQL: SELECT ... FROM "products" WHERE "id" = $1 LIMIT 1
-> returns the Product, or null if no row (throw NotFoundException yourself)

// UPDATE
await this.productRepo.update(1, {
  price: 45000,
});
SQL: UPDATE "products" SET "price" = $1 WHERE "id" = $2
-> returns { affected: 1 } — use affected === 0 to detect "not found"

// DELETE
await this.productRepo.delete(1);
SQL: DELETE FROM "products" WHERE "id" IN ($1)
-> returns { affected: 1 }
```

NOTES:

- save({...}) with no id = INSERT; save() of an object WITH an id = UPDATE
- update()/delete() run one direct statement; they don't return the row. To return the updated row: findOneBy() after update(), or use preload() + save() (theory/41 STEP 9).

2. FILTER PRODUCTS

```
import { MoreThan } from 'typeorm';

// Products costing more than 1000
await this.productRepo.find({
  where: {
    price: MoreThan(1000),
  },
});
SQL: SELECT ... FROM "products" WHERE "price" > $1     -- [1000]
```

More filters with the same pattern:

```
import { LessThan, Between, ILike, In, Not, IsNull } from 'typeorm';
```

where: { price: LessThan(500) }	price < 500	
where: { price: Between(1000, 5000) }	price BETWEEN 1000 AND 5000	
where: { name: ILike('%phone%') }	name ILIKE '%phone%'	(any case)
where: { id: In([1, 2, 3]) }	id IN (1, 2, 3)	
where: { name: Not('Laptop') }	name != 'Laptop'	
where: { category: IsNull() }	categoryId IS NULL	

```
// AND = several fields in one object
where: { name: ILike('%laptop%'), price: MoreThan(30000) }

// OR = array of objects
where: [{ price: LessThan(500) }, { name: 'Laptop' }]

// with sorting + paging
await this.productRepo.find({
  where: { price: MoreThan(1000) },
  order: { price: 'ASC' },
  skip: 0,
  take: 10,
});
SQL: ... WHERE "price" > $1 ORDER BY "price" ASC LIMIT 10 OFFSET 0
```

3. COMPLEX QUERY USING QUERYBUILDER

Find laptops costing more than 30,000, sorted by price, max 10 results.

```
await this.productRepo
  .createQueryBuilder('product')            // alias "product" for the products table
  .where('product.name ILIKE :name', {
    name: '%laptop%',
  })
  .andWhere('product.price > :price', {
    price: 30000,
  })
  .orderBy('product.price', 'DESC')
  .take(10)
  .getMany();

SQL: SELECT "product"."id", "product"."name", "product"."price"
     FROM "products" "product"
     WHERE "product"."name" ILIKE $1 AND "product"."price" > $2
     ORDER BY "product"."price" DESC
     LIMIT 10                                -- ['%laptop%', 30000]
```

Line by line:

createQueryBuilder('product')	start a SELECT on products, alias "product"
.where(...)	first condition
.andWhere(...)	add AND condition (.orWhere for OR)
:name / :price	named parameters → become $1, $2 (safe)
.orderBy(col, 'DESC')	ORDER BY
.take(10)	LIMIT 10 (.skip(n) for OFFSET)
.getMany()	run it, return Product[]

Ending methods:

```
  getMany()         Product[]
  getOne()          Product | null
  getManyAndCount() [Product[], total]
  getCount()        number
  getRawMany()      plain rows (for COUNT/SUM/aliases)

The SAME query with find options (no QueryBuilder needed here):
  await this.productRepo.find({
    where: { name: ILike('%laptop%'), price: MoreThan(30000) },
    order: { price: 'DESC' },
    take: 10,
  });
QueryBuilder becomes worth it when conditions are added only if the
user sent them, or for joins/aggregates (theory/46 Section 2).
```

4. JOIN TWO TABLES

Each product belongs to a category (the @ManyToOne in SETUP is REQUIRED — without the relation TypeORM doesn't know how to join).

```
await this.productRepo
  .createQueryBuilder('product')
  .leftJoinAndSelect('product.category', 'category')
  .getMany();

SQL: SELECT "product"."id", "product"."name", "product"."price",
            "category"."id", "category"."name"
     FROM "products" "product"
     LEFT JOIN "categories" "category"
            ON "category"."id" = "product"."categoryId"
```

Result:

```
[
  { id: 1, name: 'Laptop', price: 50000, category: { id: 1, name: 'Electronics' } },
  { id: 2, name: 'Pen',    price: 20,    category: null }       // LEFT JOIN keeps it
]
```

'product.category'	the relation property on Product (not the table name)
'category'	alias for the joined table

Join types:

leftJoinAndSelect	all products; category = null if none
innerJoinAndSelect	only products that HAVE a category
leftJoin	join to filter on it, but don't load its columns

Filter on the joined table:

```
await this.productRepo
  .createQueryBuilder('product')
  .innerJoinAndSelect('product.category', 'category')
  .where('category.name = :cat', { cat: 'Electronics' })
  .orderBy('product.price', 'ASC')
  .getMany();
```

Same simple join WITHOUT QueryBuilder:

```
  await this.productRepo.find({ relations: { category: true } });

Count products per category (join + aggregate):
  await this.productRepo
    .createQueryBuilder('product')
    .leftJoin('product.category', 'category')
    .select('category.name', 'category')
    .addSelect('COUNT(product.id)', 'total')
    .addSelect('AVG(product.price)', 'avgPrice')
    .groupBy('category.name')
    .getRawMany();
  -> [{ category: 'Electronics', total: '3', avgPrice: '41666.67' }]
     (COUNT/AVG come back as strings from Postgres)
```

Create a product WITH a category:

```
await this.productRepo.save({
  name: 'Phone',
  price: 20000,
  category: { id: 1 },       // just the id is enough -> categoryId = 1
});
```

SAME CODE, ANY POSTGRES HOST

```
Only .env (and ssl) change; every query above stays identical:

  Docker     DATABASE_URL="postgresql://postgres:devpass@localhost:5432/shop"     ssl: false
  Local      DATABASE_URL="postgresql://postgres:mypass@localhost:5432/shop"      ssl: false
  Supabase   DATABASE_URL="postgresql://postgres.<ref>:<pass>@aws-0-
<region>.pooler.supabase.com:5432/postgres"
                                                                                  ssl: {
```

```
rejectUnauthorized: false }
  Neon       DATABASE_URL="postgresql://<user>:<pass>@ep-xxx.neon.tech/neondb?sslmode=require"
```

That's the point of an ORM + env vars: the database location is configuration, the queries are code.

QUICK REFERENCE

Task	TypeORM
create	save({ ... })
all rows	find()
by id	findOneBy({ id })
update	update(id, { ... })
delete	delete(id)
filter	find({ where: { price: MoreThan(1000) } })
sort + page	find({ order, skip, take })
search, dynamic, complex	createQueryBuilder().where().andWhere()...
join	relations: { category: true } or leftJoinAndSelect
group / count / sum	createQueryBuilder().select().groupBy().getRawMany()
very complex report	dataSource.query('SELECT ...', [params])

★ RULE OF THUMB

Repository methods for CRUD and simple filters, QueryBuilder for search/complex conditions and joins, raw SQL only for heavy reports. Joins need a relation (@ManyToOne / @OneToMany) on the entity. And the queries never change between Docker, Supabase or any other Postgres — only DATABASE_URL does.

See also: **theory/46-which-query-tool-to-use-detailed.txt** (each tool in depth), **theory/45-typeorm-vs-writing-sql.txt**, **theory/43-where-sql-queries-come-from.txt**, **theory/42-ways-to-connect-postgres.txt** (hosts), **theory/44-how-the-table-was-created.txt** (entities → tables)